Library Management System Report

1. **Title Page:**

INSTITUTE OF ENGINEERING AND MANAGEMENT

COURSE NAME: ADVANCED PROGRAMMING (OOP)

PROJECT TITLE: LIBRARY MANAGEMENT SYSTEM

STUDENT NAME: AUGNIK KUMAR SAHOO

ROLL NUMBER: 54

INSTRUCTOR NAME: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

SEMESTER: SPRING 2026

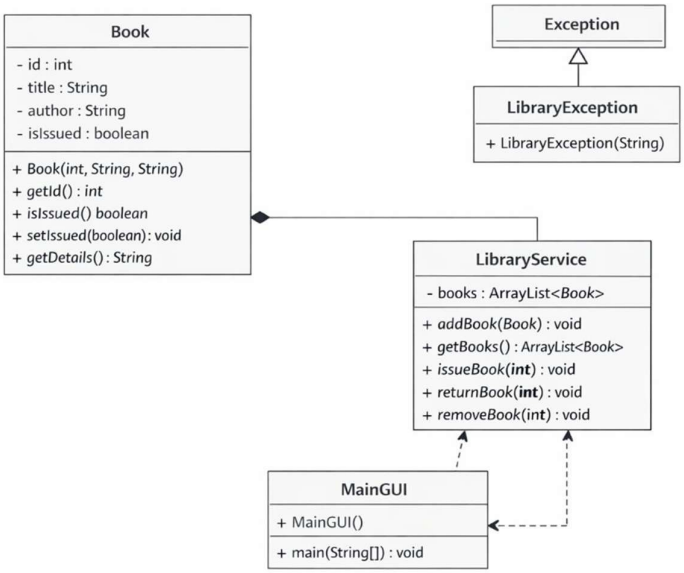
2. Problem Statement:

Managing library records manually is difficult and time-consuming, especially when handling a large number of books and tracking their issue and return status. A digital system is required to efficiently organize book information and minimize human errors.

This project implements a Library Management System using Java with a graphical user interface. It allows users to add, view, issue, return, and remove books. The system uses Object-Oriented Programming concepts and Java collections to manage data efficiently. The modular design improves readability and maintainability.

3. System Design:

3.1 UML Class Diagram:



3.2 Description of Major Classes:

- Book Class: Represents a book entity with attributes like id, title, author, and issue status.
- LibraryService Class: Handles all operations such as adding, issuing, returning, and removing books.
- LibraryException Class: Custom exception class used for handling errors like book not found.
- MainGUI Class: Provides a graphical user interface using Java Swing for user interaction.

3.2 Package Structure:

- model → Book class
- service → LibraryService class
- util → LibraryException class
- main → MainGUI class

3.3 Interaction Flow:

1. User clicks a button in GUI
2. Input is taken using dialog box
3. Request sent to LibraryService
4. Service processes logic
5. Result shown via popup

4. OOP Concepts Demonstrated:

4.1 Encapsulation:

Where: Book class

Why: To protect data from direct access

Snippet:

```
private int id;
private String title;
public int getId() {
 return id;
 }
```

Explanation: Data is hidden and accessed via methods.

4.2 Abstraction:

Where: LibraryService

Why: To hide internal logic

Snippet:

```
public void addBook(Book book) {
  books.add(book);
  }
```

Explanation: User does not know how books are stored.

4.3 Inheritance:

Where: LibraryException

Why: To reuse exception functionality

Snippet:

```
public class LibraryException extends Exception {
public LibraryException(String message) {
super(message);
  }
}
```

Explanation: Custom exception inherits from Exception class.

4.4 Polymorphism:

Where: Exception handling

Why: Different behavior at runtime

Snippet:

```
catch (LibraryException e) {
System.out.println(e.getMessage());
 }
```

Explanation: Different messages handled dynamically.

5. Implementation Highlights:

5.1 Adding Books:

```
library.addBook(new Book(id, title, author));
```

Justification: Creates and stores new book object.

5.2 Issuing Books:

```
if (b.isIssued()) {
  throw new LibraryException("Book already issued!");
```

Justification: Prevents duplicate issuing.

5.3 GUI Interaction:

JOptionPane.showInputDialog("Enter ID:");

Justification: Enables user-friendly input.

6. Testing & Error Handling:

- Test Cases:

I. Add book
II. View books
III. Issue book
IV. Return book
V. Remove book

- Edge Cases:

I. Issuing already issued book
II. Returning non-issued book

- Failure Scenarios

I. Invalid ID input
II. Book not found

7. Git Workflow:

- Initialized repository
- Added initial project structure
- Implemented model class
- Added service logic
- Implemented GUI
- Fixed bugs and tested

8. Conclusion & Future Scope:

The project successfully demonstrates a Library Management System using Java with GUI and OOP concepts. It provides an efficient way to manage library operations.

Future Scope:

- Add database integration
- Improve GUI design
- Add user authentication
- Implement search functionality

Appendix:

1. Copy of the initial project proposal

Project Title: Library Management System with Proper Abstraction and Modular Structure

Problem Statement:
Managing library records manually can be difficult and time-consuming, especially when handling many books and tracking their issue and return status. A digital system is needed to organize book information efficiently and reduce errors. The proposed **Library Management System** will allow users to add, view, issue, return, and remove books using a structured Java application. The system will apply **Object-Oriented Programming concepts** and collections to manage library data in a simple and efficient way.

Target User:

- Librarians managing library records
- Students accessing library books
- Educational institutions and small libraries
- Library administrators

Core Features:

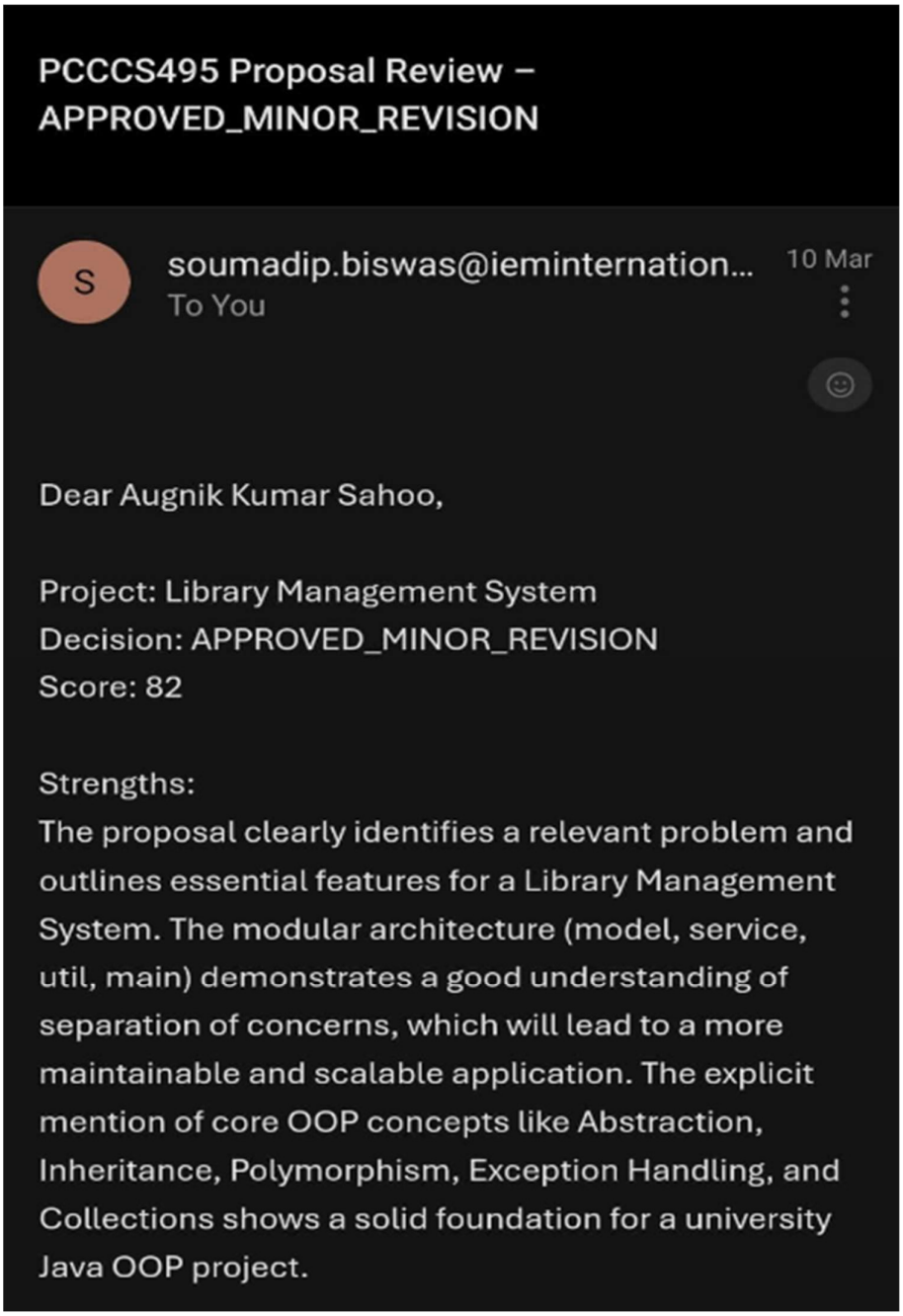
- Add new books to the library
- View all available books
- Issue books to users, Return issued books
- Remove books from the system
- Display book details such as ID, title, author, and issue status

OOP Concepts to be Used:

- **Abstraction**: Library operations such as adding, issuing, and returning books are handled through service classes, hiding internal implementation details from the user.
- **Inheritance**: Classes can extend base classes (for example, different types of users or library items) to reuse common properties and behaviors.
- **Polymorphism**: Method overriding or interface implementation allows different classes to provide their own implementation of common operations.
- **Exception Handling**: Try–catch blocks and custom exceptions will handle errors such as issuing unavailable books or accessing non-existent records.
- **Collections / Threads (if applicable)**: Java **ArrayList** collection will be used to store and manage multiple book objects dynamically.

Proposed Architecture Description:
The system will follow a **modular architecture** where different components of the program are separated into packages such as **model, service, util, and main**. The **model layer** represents the book data structure, the **service layer** handles library operations like adding or issuing books, the **utility layer** manages custom exceptions, and the **main layer** controls the user interface and program execution. This modular design improves readability, maintainability, and scalability of the application while clearly separating responsibilities among different components of the system.

2. Copy of the review mail received (with APPROVED_MINOR)



PCCCS495 Proposal Review – APPROVED_MINOR_REVISION

S soumadip.biswas@ieminternation... 10 Mar
To You

Dear Augnik Kumar Sahoo,

Project: Library Management System
Decision: APPROVED_MINOR_REVISION
Score: 82

Strengths:
The proposal clearly identifies a relevant problem and outlines essential features for a Library Management System. The modular architecture (model, service, util, main) demonstrates a good understanding of separation of concerns, which will lead to a more maintainable and scalable application. The explicit mention of core OOP concepts like Abstraction, Inheritance, Polymorphism, Exception Handling, and Collections shows a solid foundation for a university Java OOP project.